

Inżynieria pozyskiwania i ochrony wiedzy z danych i baz danych

Obfuskator i detektor obfuskacji kodu źródłowego programów

Raport 1

Rafał Starypan, Eryk Mika, Kamil Wojcieszak

1. Wprowadzenie

Obfuskacja kodu źródłowego (ang. code obfuscation) to proces transformacji programu do postaci funkcjonalnie równoważnej, lecz znacząco trudniejszej do zrozumienia i analizy przez człowieka lub narzędzia automatyczne. Technika ta jest powszechnie stosowana w celu ochrony własności intelektualnej, zabezpieczenia algorytmów przed inżynierią wsteczną (reverse engineering), a także w kontekście ochrony licencji oprogramowania.

Jednocześnie te same metody obfuskacji wykorzystywane są przez twórców złośliwego oprogramowania w celu ukrycia szkodliwego kodu przed skanerami antywirusowymi i systemami detekcji. Z tego powodu rozwój technik zarówno obfuskacji, jak i jej wykrywania, stanowi istotny obszar badawczy w dziedzinie bezpieczeństwa informatycznego.

Niniejszy raport stanowi przegląd literatury naukowej oraz istniejących rozwiązań w zakresie obfuskacji i detekcji obfuskacji kodu źródłowego. Dokument ten został przygotowany w ramach punktu kontrolnego nr 2 projektu semestralnego.

2. Klasyfikacja technik obfuskacji

Techniki obfuskacji kodu można podzielić na kilka głównych kategorii, w zależności od poziomu, na którym dokonywana jest transformacja. Poniższa taksonomia opiera się na pracach Collberga i in. (1997) oraz nowszych przeglądach systematycznych, takich jak Xu i in. (2020).

2.1. Obfuskacja układu (Layout Obfuscation)

Obfuskacja na poziomie układu kodu polega na modyfikacji elementów, które nie wpływają na semantykę programu, ale znacząco utrudniają jego czytelność. Do tej kategorii należą następujące techniki:

Zmiana nazw identyfikatorów (identifier renaming) – zamiana nazw zmiennych, funkcji i klas na bezsensowne ciągi znaków. Jest to najprostsza i najczęściej stosowana forma obfuskacji, implementowana m.in. przez narzędzie ProGuard dla języka Java/Android.

Usuwanie symboli debugowania (stripping) – pozbycie się informacji diagnostycznych, komentarzy i metadanych z kodu wynikowego.

Wstawianie kodu śmieciowego (junk code insertion) – dodawanie instrukcji, które nie mają wpływu na wynik działania programu, lecz zwiększają jego rozmiar i złożoność.

Separacja kodu powiązanego logicznie – rozmieszczanie fragmentów realizujących wspólne zadanie w odległych częściach pliku źródłowego.

2.2. Obfuskacja przepływu sterowania (Control Flow Obfuscation)

Techniki tej grupy zmieniają strukturę grafu przepływu sterowania programu, zachowując jego zachowanie. Do najważniejszych należą:

Splaszczanie przepływu sterowania (control flow flattening) – zastąpienie naturalnej struktury kodu (pętle, warunki) jedną wielką instrukcją switch sterowaną zmienną stanu. Jest to jedna z najbardziej efektywnych technik obfuskacji, implementowana m.in. w Obfuscator-LLVM oraz Tigress.

Predykaty nieprzezroczyste (opaque predicates) – wyrażenia warunkowe, których wartość jest znana w czasie kompilacji, ale trudna do ustalenia przez analizator. Pozwalają wprowadzać fałszywe rozgałęzienia w kodzie.

Fałszywy przepływ sterowania (bogus control flow) – wstawianie dodatkowych bloków kodu z warunkami opartymi na predykatkach nieprzezroczystych, które nigdy nie zostaną wykonane.

2.3. Obfuskacja danych (Data Obfuscation)

Techniki ukierunkowane na transformację struktur danych używanych przez program:

Szyfrowanie łańcuchów znaków (string encryption) – przechowywanie stałych tekstowych w postaci zaszyfrowanej i deszyfrowanie ich w czasie wykonywania. Popularna metoda stosowana m.in. przez DashO.

Transformacja tablic – dzielenie, scalanie, zmiana wymiarowości tablic (folding/flattening) oraz zmiana kolejności elementów w celu zmniejszenia lokalności danych.

Arytmetyka mieszana logiczno-arytmetyczna (Mixed Boolean-Arithmetic, MBA) – zastępowanie prostych wyrażeń arytmetycznych funkcjonalnie równoważnymi, ale znacznie bardziej złożonymi wyrażeniami łączącymi operacje logiczne i arytmetyczne.

2.4. Zaawansowane techniki

Wirtualizacja kodu (code virtualization) – przekształcenie fragmentu programu w interpreter z własnym zestawem instrukcji bajtowych (bytecode). Jest to jedna z najsilniejszych znanych technik obfuskacji. Narzędzie Tigress oferuje zaawansowaną wirtualizację z możliwością generowania interpreterów o zróżnicowanej złożoności.

Samomodyfikujący się kod (self-modifying code) – kod, który zmienia własne instrukcje w czasie wykonywania, utrudniając zarówno analizę statyczną, jak i dynamiczną.

Pakowanie (packing) – kompresja skompilowanego programu do postaci, która rozpakowuje się w pamięci w czasie uruchamiania. Narzędzie UPX jest przykładem popularnego pakera.

3. Metody detekcji obfuskacji

Detekcja obfuskacji jest kluczowym elementem analizy bezpieczeństwa oprogramowania. Współczesne podejścia można podzielić na trzy główne kategorie.

3.1. Analiza statyczna

Analiza statyczna polega na badaniu kodu bez jego uruchamiania. Cechy, które mogą wskazywać na obfuskację, obejmują: wysoką entropię kodu i łańcuchów znaków, nietypowe nazwy identyfikatorów (losowe ciągi, bardzo krótkie nazwy), anomalnie wysoką złożoność cykloatmatyczną, obecność martwego kodu oraz nietypową strukturę grafu przepływu sterowania.

3.2. Analiza dynamiczna

Analiza dynamiczna polega na obserwacji zachowania programu w trakcie jego wykonywania. Pozwala wykryć techniki obfuskacji, które są trudne do zidentyfikowania statycznie, takie jak: rozpakowywanie kodu w pamięci, deszyfrowanie łańcuchów w czasie wykonywania, samomodyfikacja kodu czy wirtualizacja. Narzędzia takie jak symboliczne silniki wykonawcze (np. KLEE) i instrumentacja dynamiczna (np. Pin, DynamoRIO) są powszechnie stosowane.

3.3. Podejścia oparte na uczeniu maszynowym

Nowoczesne badania coraz częściej wykorzystują metody uczenia maszynowego do detekcji i klasyfikacji obfuskacji. Poniższa tabela podsumowuje kluczowe prace w tym obszarze.

Praca / Narzędzie	Metoda ML	Cel detekcji	Wyniki
Conti i in. (2022)	Deep learning (CNN, NLP, model hybrydowy)	Obfuskacja aplikacji Android	F-measure: 0.985
Adobe SI (Tang, 2021)	Konwolucyjne sieci neuronowe na poziomie znaków	Obfuskowane komendy systemowe	F1: 0.989
Raitsis i in. (2025)	Random Forest, Gradient Boosting, SVM	Pliki EXE i BAT (Jlaive, Pyarmor, itp.)	Accuracy: 97.3%
ALMOND (2025)	Model językowy (zero-shot, pre-trained na assembly)	Obfuskacja plików binarnych	Acc: 0.948 (ET)
Wermke i in. (2016)	Klasyfikator ML (cechy statyczne bytecode)	Obfuskacja leksykalna i stringowa w Java malware	Accuracy: 99%
Klasyfikacja w LLVM IR (2025)	ExtraTrees, CatBoost na wektorach IR2Vec	Techniki obfuskacji Tigress	Acc: 0.948

Kluczowe cechy (features) wykorzystywane do detekcji obfuskacji obejmują: entropię kodu i łańcuchów znaków, statystyki długości tokenów, gęstość komentarzy, stosunek unikalnych tokenów, liczbę słów kluczowych, a także reprezentacje wektorowe kodu pośredniego (np. IR2Vec w kontekście LLVM IR).

4. Przegląd istniejących narzędzi

4.1. Narzędzia obfuskacji

Narzędzie	Język	Główne techniki	Typ	Licencja
Tigress	C (C99)	Wirtualizacja, CFF, opaque predicates, junk code, MBA	Source-to-source	Darmowy (closed-source)
Obfuscator-LLVM	C, C++, Obj-C, Fortran	CFF, substytucja instrukcji, bogus CF	Kompilator (LLVM IR)	Open-source
ProGuard	Java / Android	Renaming, shrinking, optymalizacja	Bytecode	Open-source
Pyarmor	Python	Szyfrowanie bytecode, ograniczenia uruchamiania	Bytecode	Komercyjny
javascript-obfuscator	JavaScript	CFF, string encryption, renaming, dead code	Source-to-source	Open-source
UPX	Pliki binarne	Kompresja/pakowanie plików wykonywalnych	Paker	Open-source

4.2. Narzędzia detekcji i deobfuskacji

Narzędzie	Opis	Zastosowanie
Adobe SI Obfuscation Detection	Pakiet Python oparty na głębokim CNN do wykrywania obfuskowanych komend	Analiza bezpieczeństwa systemów
KLEE	Symboliczny silnik wykonawczy umożliwiający analizę obfuskowanych binarek	Deobfuskacja, testowanie
Triton	Framework do analizy dynamicznej z symbolicznym wykonywaniem i analizą taint	Deobfuskacja wirtualizacji (Tigress)
de4dot	Deobfuskator i depaker dla aplikacji .NET	Analiza malware .NET
JEB / Jadx	Dekompilatory z wbudowaną obsługą deobfuskacji dla Androida	Reverse engineering APK

5. Algorytmy relewantne dla projektu

Na podstawie przeglądu literatury, poniżej przedstawiono algorytmy i podejścia najbardziej istotne z perspektywy planowanego projektu obfuskatora i detektora obfuskacji.

5.1. Algorytmy obfuskacji do implementacji

Zmiana nazw identyfikatorów

Algorytm polega na parsowaniu drzewa składniowego (AST) programu, identyfikacji wszystkich deklaracji zmiennych, funkcji i klas, a następnie zastąpieniu ich nazw losowymi lub bezsensownymi ciągami znaków. Ważne jest zachowanie spójności – każde odwołanie do danego symbolu musi zostać zamienione na tę samą nową nazwę. Technika ta jest stosunkowo prosta w implementacji i skuteczna w utrudnianiu czytania kodu.

Wstawianie martwego kodu

Algorytm generuje losowe, ale składniowo poprawne instrukcje, które nie mają wpływu na wynik programu, i wstawia je między istniejące instrukcje. Warianty obejmują: dodawanie nieużywanych zmiennych, tworzenie pętli z zerową liczbą iteracji, wstawianie instrukcji warunkowych z predykatami zawsze fałszywymi.

Spłaszczanie przepływu sterowania

Algorytm CFF (Control Flow Flattening) przekształca graf przepływu sterowania funkcji w pętlę z instrukcją switch. Każdy blok bazowy (basic block) staje się przypadkiem (case) instrukcji switch, a zmienna sterująca (dispatcher variable) określa kolejność ich wykonywania. Implementacja wymaga: identyfikacji bloków bazowych, przypisania im unikalnych identyfikatorów, utworzenia pętli dispatch z instrukcją switch, oraz aktualizacji zmiennej sterującej na końcu każdego bloku.

Szyfrowanie stałych tekstowych

Łańcuchy znaków w kodzie źródłowym są zastępowane zaszyfrowanymi odpowiednikami, a na początku programu (lub leniwie, przy pierwszym dostępie) dodawana jest funkcja deszyfrująca. Najprostsze warianty wykorzystują operację XOR z kluczem, bardziej zaawansowane – szyfrowanie AES lub RC4.

5.2. Algorytmy detekcji do implementacji

Analiza entropii

Entropia Shannona jest jedną z podstawowych metryk wykorzystywanych do detekcji obfuskacji. Obfuskowany kod zwykle wykazuje wyższą entropię niż kod nieobfuskowany, ze względu na losowość nazw, zaszyfrowane łańcuchy i dodany szum. Detektor oblicza entropię fragmentów kodu i porównuje ją z ustalonymi progami.

Analiza struktury AST

Anomalie w drzewie AST mogą wskazywać na obfuskację: nadmiarowa głębokość zagnieżdżeń, nietypowa złożoność cykliczna, obecność dużych instrukcji switch (wskaźnik CFF) czy nierealistyczne proporcje między rozmiarem kodu a liczbą sensownych identyfikatorów.

Klasyfikacja za pomocą uczenia maszynowego

Na podstawie zestawu cech (features) wyodrębnionych z kodu – takich jak entropia, statystyki tokenów, złożoność cykliczna, gęstość komentarzy – trenowany jest model klasyfikujący kod jako obfuskowany lub nieobfuskowany. Badania wskazują, że modele Random Forest i

Gradient Boosting osiąga bardzo dobre wyniki (accuracy powyżej 97%) w zadaniach detekcji obfuskacji.

6. Przegląd literatury

- [1] **Collberg C., Thomborson C., Low D.** (1997). A Taxonomy of Obfuscating Transformations. Technical Report 148, University of Auckland. – Przełomowa praca definiująca taksonomię transformacji obfuskujących, podział na obfuskację układu, danych, przepływu sterowania i przewencyjną.
- [2] **Schrittwieser S. i in.** (2016). Protecting Software through Obfuscation: Can It Keep Pace with Progress in Code Analysis? ACM Computing Surveys, Vol. 49, No. 1. – Obszerny przegląd technik obfuskacji i odporności na metody analizy kodu.
- [3] **Xu H., Zhou Y., Kang Y., Lyu M.R.** (2020). On Secure and Usable Program Obfuscation: A Survey. arXiv:1710.01139. – Przegląd podejść do obfuskacji zarówno zorientowanych na kod, jak i na model, z analizą brakujących metryk bezpieczeństwa.
- [4] **Hosseinzadeh S. i in.** (2018). Diversification and obfuscation techniques for software security: A systematic literature review. Information and Software Technology, Vol. 104. – Systematyczny przegląd technik dywersyfikacji i obfuskacji stosowanych w ochronie oprogramowania.
- [5] **Junod P. i in.** (2015). Obfuscator-LLVM – Software Protection for the Masses. IEEE/ACM Workshop on Software Protection (SPRO). – Prezentacja narzędzia O-LLVM, pierwszego otwartego obfuskatora na poziomie LLVM IR.
- [6] **Conti M., Vinod P., Vitella A.** (2022). Obfuscation detection in Android applications using deep learning. Journal of Information Security and Applications, Vol. 69. – Modele deep learning do wykrywania obfuskacji w aplikacjach Android z F-measure 0.985.
- [7] **Raitsis T. i in.** (2025). Code Obfuscation: A Comprehensive Approach to Detection, Classification, and Ethical Challenges. Algorithms, 18(2), 54. – Praktyczne podejście do detekcji obfuskacji z wykorzystaniem RF, GB i SVM.
- [8] **ALMOND** (2025). Learning an Assembly Language Model for 0-Shot Code Obfuscation Detection. Proceedings of the ACM on Software Engineering. – Nowatorskie podejście zero-shot do detekcji obfuskacji za pomocą modelu językowego wytrenowanego na nieobfuskowanym kodzie assembly.
- [9] **Banescu S., Collberg C., Pretschner A.** (2017). Predicting the Resilience of Obfuscated Code Against Symbolic Execution Attacks via Machine Learning. USENIX Security. – Zastosowanie ML do przewidywania odporności obfuskacji na ataki symbolicznego wykonywania.
- [10] **Xu H. i in.** (2020). Layered obfuscation: a taxonomy of software obfuscation techniques for layered security. Cybersecurity, Springer. – Nowa taksonomia technik obfuskacji w kontekście warstwowego bezpieczeństwa.